Trabajo Práctico Integrador – Programación II

Sistema SmartGrids | Segunda Parte

Integrante:
Tomás Granillo , Tomi Luna

Materia: Programación II

Comisión: com-05
Año: 2026

1. Introducción

El presente informe documenta la implementación de la segunda parte del Trabajo Práctico Integrador de Programación II. El sistema SmartGrids permite registrar y consultar plantas de generación eléctrica, almacenar sus monitoreos, controlar sus estados de actividad y obtener estadísticas sobre la energía generada y el cumplimiento de las cuotas.

La solución se organiza en una clase principal SmartGrids y una jerarquía de clases para representar los distintos tipos de plantas y registros de monitoreo. Esta organización permite reutilizar atributos y comportamientos comunes y especializar las reglas de generación según la tecnología de cada planta.

2. Diseño y organización de clases

SmartGrids implementa la interfaz ISmartGrids y mantiene las plantas en un Map<String, Planta>, concretamente un LinkedHashMap. La clave es el nombre de la planta y el valor es el objeto Planta correspondiente. Cada planta mantiene sus monitoreos en un Map<LocalDateTime, RegistroMonitoreo>, implementado mediante HashMap, evitando dos registros con la misma fecha y hora.

La jerarquía de plantas parte de Planta, que es abstracta. PlantaRenovable también es abstracta y extiende Planta. PlantaSolar, PlantaEolica y PlantaHidraulica extienden PlantaRenovable. PlantaTermica extiende directamente Planta.

Los registros de monitoreo parten de RegistroMonitoreo, clase abstracta. MonitoreoGeneral, MonitoreoEolico y MonitoreoTermico extienden esa clase y agregan, cuando corresponde, los datos particulares de cada tecnología. PeriodoInactividad representa un intervalo entre una fecha de desactivación y una fecha de reactivación.

3. Aplicación de herencia, polimorfismo y abstracción

Herencia: se aplica en la jerarquía Planta → PlantaRenovable → PlantaSolar/PlantaEolica/PlantaHidraulica, y en Planta → PlantaTermica. También se aplica en RegistroMonitoreo → MonitoreoGeneral/MonitoreoEolico/MonitoreoTermico. Las subclases reutilizan los atributos y operaciones comunes definidos en sus superclases y agregan los propios.

Abstracción: Planta y RegistroMonitoreo son clases abstractas porque representan conceptos generales y no se instancian directamente. Planta declara como abstractos capacidadMinimaCuota() y cumpleCuota(RegistroMonitoreo), que deben resolverse en cada tipo concreto. PlantaRenovable agrupa características compartidas por las plantas solar, eólica e hidráulica.

Polimorfismo: SmartGrids trabaja con referencias del tipo general Planta y RegistroMonitoreo, aunque en ejecución los objetos sean de una subclase concreta. Por ejemplo, en histogramaCumplimiento() se invoca planta.cumpleCuota(monitoreo). La implementación ejecutada depende del tipo real de planta: solar, eólica, hidráulica o térmica. De este modo, el algoritmo general no necesita conocer en detalle todas las fórmulas particulares.

También hay polimorfismo al recorrer plantas.values() como elementos Planta y al recorrer planta.getMonitoreos().values() como RegistroMonitoreo, permitiendo procesar objetos de distintas subclases mediante un tipo común.

4. Sobreescritura, sobrecarga e interfaces

Sobreescritura (override): las clases PlantaSolar, PlantaEolica, PlantaHidraulica y PlantaTermica redefinen capacidadMinimaCuota() y cumpleCuota(RegistroMonitoreo), heredados como métodos abstractos de Planta. Cada una implementa su regla específica. También se sobreescribe toString() en PlantaRenovable y en las plantas concretas para agregar información propia; MonitoreoEolico y MonitoreoTermico también redefinen toString().

Sobrecarga (overload): SmartGrids define tres métodos registrarMonitoreo con el mismo nombre y distintos parámetros: uno para monitoreo general, otro que recibe velocidadViento y otro que recibe co2Emitido y combustibleUtilizado. El compilador selecciona la versión según la cantidad y los tipos de argumentos.

Interfaces: SmartGrids implements ISmartGrids. La interfaz establece el contrato público de operaciones que debe ofrecer el sistema, mientras que SmartGrids aporta su implementación. Según el enunciado, se utiliza así el concepto de interfaces además de herencia, polimorfismo, abstracción, sobreescritura y sobrecarga.

5. Uso de herramientas y estructuras de Java

StringBuilder: se utiliza para construir textos que se van modificando, especialmente en horasSinVelocidadMinimaPorPlantaEolica() y en toString() de SmartGrids. Es apropiado porque se agregan fragmentos de texto durante los recorridos.

Foreach: se utiliza para recorrer colecciones, por ejemplo al consultar plantas, sumar energía, analizar monitoreos y construir estadísticas.

Iteradores: en la implementación revisada no se identificó un uso explícito de Iterator. El enunciado exige utilizar al menos una vez Iteradores y Foreach; por lo tanto, antes de entregar debe incorporarse y verificarse un recorrido explícito con Iterator si aún no existe en otra parte del proyecto.

LinkedHashMap conserva el orden de inserción de las plantas. HashMap permite buscar monitoreos por fecha y hora. HashSet evita contar dos veces un mismo día para una hora del histograma. ArrayList almacena resultados y elementos ordenables.

6. IREP de los TADs

SmartGrids: el mapa de plantas está inicializado; cada clave coincide con el nombre de la planta almacenada; no hay nombres repetidos; cada valor es una planta válida. Los monitoreos pertenecen a una única planta y se identifican por fecha y hora.

Planta: nombre y dirección no nulos ni vacíos; capacidad máxima positiva; fecha de puesta en funcionamiento definida; estado de actividad coherente; mapa de monitoreos inicializado; no hay dos monitoreos para la misma fecha y hora; los monitoreos no son anteriores a la puesta en funcionamiento. Cada período de inactividad tiene un inicio definido y, si está cerrado, una fecha de fin no anterior al inicio.

PlantaRenovable: cumple el IREP de Planta; fuente primaria no nula ni vacía; eficiencia promedio dentro del intervalo [0, 1].

PlantaSolar: cumple el IREP de PlantaRenovable; superficie cubierta positiva; cantidad de paneles positiva; horas de radiación diaria entre 0 y 24.

PlantaEolica: cumple el IREP de PlantaRenovable; cantidad de aerogeneradores positiva; altura de torres positiva; velocidad mínima de viento no negativa.

PlantaHidraulica: cumple el IREP de PlantaRenovable; cantidad de turbinas positiva; capacidad del embalse positiva.

PlantaTermica: cumple el IREP de Planta; tipo de combustible no nulo ni vacío; objetivo de emisiones no negativo; capacidad de almacenamiento de combustible positiva.

RegistroMonitoreo: fecha y hora definidas; energía generada no negativa; cantidad de incidencias no negativa.

MonitoreoGeneral: cumple el IREP de RegistroMonitoreo y no agrega atributos específicos.

MonitoreoEolico: cumple el IREP de RegistroMonitoreo; velocidad del viento no negativa; se utiliza para una planta eólica.

MonitoreoTermico: cumple el IREP de RegistroMonitoreo; CO2 emitido y combustible utilizado no negativos; se utiliza para una planta térmica.

PeriodoInactividad: fecha de inicio definida; fecha de fin nula mientras el período está abierto o no anterior a la fecha de inicio cuando está cerrado. Un período representa el intervalo [inicio, fin), por lo que el instante de reactivación ya no se considera inactivo.

7. Análisis de complejidad: punto 7 – Histograma de cumplimiento de cuota por hora

El método histogramaCumplimiento(nombre, fechaDesde, fechaHasta) devuelve un arreglo de 24 posiciones. Cada posición representa una hora del día y contiene la cantidad de días distintos, dentro del rango indicado, en los que la planta cumplió su cuota a esa hora.

Sea M la cantidad de monitoreos almacenados para la planta. El método realiza primero la búsqueda de la planta en LinkedHashMap. Esta búsqueda es O(1) promedio. El análisis siguiente considera el costo del recorrido de los M monitoreos.

Se inicializa un arreglo de tamaño 24 y un HashMap con 24 entradas, cada una asociada a un HashSet vacío. Como 24 es constante, esta inicialización cuesta O(24) = O(1).

Después se recorren los M monitoreos una sola vez. Para cada monitoreo se obtiene su fecha, se verifica si está dentro del rango y, si corresponde, se evalúa planta.cumpleCuota(monitoreo). Esa evaluación es O(1), ya que cada implementación realiza una cantidad fija de operaciones y accesos a atributos. Si cumple, se inserta el día en el conjunto correspondiente a su hora. La inserción en HashSet tiene costo O(1) promedio.

Finalmente se recorren las 24 horas y se consulta el tamaño de cada conjunto. Este paso cuesta O(24) = O(1). Por lo tanto, usando Álgebra de Órdenes: T(M) = O(1) + O(24) + M·O(1) + O(24) = O(M + 24) = O(M).

Complejidad espacial: se utiliza un arreglo de 24 enteros y 24 conjuntos. Si D es la cantidad de pares distintos (hora, día) que cumplen la cuota en el intervalo, el espacio auxiliar es O(24 + D) = O(D). En el peor caso D no supera M, por lo que el espacio auxiliar es O(M).

Nota: el análisis supone el costo promedio constante de las operaciones de HashMap y HashSet. En el peor caso teórico de colisiones, las operaciones hash pueden degradarse. Además, el rango de fechas no genera un ciclo por cada día: el método recorre los monitoreos existentes, por lo que el costo depende de M y no de la cantidad total de días del intervalo.

8. Código analizado del punto 7

El siguiente es el método de SmartGrids que calcula el histograma. Se incluye porque el enunciado solicita pegar el código de todas las funciones necesarias para justificar el análisis de complejidad.

```
@Override
public int[] histogramaCumplimiento(
        String nombre,
        String fechaDesde,
        String fechaHasta) {

    Planta planta = buscarPlanta(nombre);

    LocalDate desde = parsearFecha(fechaDesde);
    LocalDate hasta = parsearFecha(fechaHasta);

    if (hasta.isBefore(desde)) {
        throw new RuntimeException(
            "La fecha final no puede ser anterior a la fecha inicial"
        );
    }

    int[] histograma = new int[24];

    Map<Integer, Set<LocalDate>> diasPorHora = new HashMap<>();

    for (int hora = 0; hora < 24; hora++) {
        diasPorHora.put(hora, new HashSet<>());
    }

    for (RegistroMonitoreo monitoreo :
            planta.getMonitoreos().values()) {

        LocalDateTime fecha = monitoreo.getFechaHora();
```

```
            LocalDate dia = fecha.toLocalDate();

            if (dia.isBefore(desde) || dia.isAfter(hasta)) {
                continue;
            }

            if (planta.cumpleCuota(monitoreo)) {
                diasPorHora.get(fecha.getHour()).add(dia);
            }
        }

        for (int hora = 0; hora < 24; hora++) {
            histograma[hora] = diasPorHora.get(hora).size();
        }

        return histograma;
    }
```

9. Conclusión

La implementación utiliza una jerarquía de clases que separa los aspectos comunes de las plantas y de los monitoreos de las particularidades de cada tecnología. La abstracción y el polimorfismo permiten centralizar operaciones en SmartGrids y delegar las reglas específicas en las clases concretas. Además, se utilizan sobreescritura, sobrecarga e interfaces, junto con estructuras de datos adecuadas para las consultas principales.

Como tareas finales, se recomienda ajustar el formato del informe eólico, revisar la ordenación del mapa de fuentes, proteger la colección interna de monitoreos, incorporar un uso explícito de Iterator y verificar la ejecución de todos los tests y del Código Cliente.